COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 | Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	October 13, 2026	**DATE SUBMITTED:**	October 20, 2026
NAME:	Niña Denise M. Santiago Troy Jerron Bautista	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HANDS-ON ACTIVITY NO. __12.1__ **ALGORITHMIC STRATEGIES**

1. PROCEDURE OUTPUT

Procedure Output

Strategy	Algorithm	Analysis
Recursion	Pre-Order Tree Traversal	Pre-Order Tree is one of tree traversals that involves a recursion method when calling the function. In the pre-order function, it states that as long as the left and right nodes are not equal to null, it will loop back to the function until the whole tree has been traversed.
Brute Force	Linear Search Algorithm	Linear Search uses a brute force strategy to search for the key. In its function, it will compare the number to the key, basically brute forcing the list until the key has been found or the whole list has been searched, which failed to find the key.
Backtracking	DFS Graph Traversal	The DFS Graph traversal uses a

		backtracking strategy to find traversal to every single edge. In the DFS function, it first starts at a chosen value and continues to traverse to the left side of the node, but when it can't continue further, it goes to the right and explores that side, until the whole graph has been traversed.
Greedy	Selection Sort	Section sort has a greedy-like approach when sorting a list. Selection sort repeatedly finds the smallest element in the unsorted position and places it the correction spot without considering the previously sorted values.
Divide-and-Conquer	Merge Sort	A merge sort algorithm is an example of a divide-and-conquer strategy to sort list. In merge sort, it divides the list into smaller parts, sort those parts, and conquer by merging with other small parts, until it merges into one list and has been sorted properly.

Table 12-1. Algorithmic Strategies and Examples

Screenshot

ILO_B_Memoization.cpp > ...

```cpp
#include <iostream>
#include <algorithm>
#include <vector>

const int MAXN = 100005;
std::vector<int> memo(MAXN, -1); // Global memo table initialized to -1

int getMinSteps ( int n )
{
    using std::min;
    if ( n == 1 ) return 0; // base case
    if ( memo[n] != -1 ) return memo[n]; // we have solved it already :)
    int r = 1 + getMinSteps( n-1 ); // '-1' step. 'r' will contain the optimal answer finally
    if ( n%2 == 0 ) r = min( r , 1 + getMinSteps( n / 2 ) ); // '/2' step
    if ( n%3 == 0 ) r = min( r , 1 + getMinSteps( n / 3 ) ); // '/3' step
    memo[n] = r ; // save the result. If you forget this step, then its same as plain recursion.
    return r;
}

int main() {
    int n = 10;
    std::cout << "Minimum steps to reach " << n << " is: " << getMinSteps(n) << std::endl;
    return 0;
}
```

Output

```
Minimum steps to reach 10 is: 3
```

Analysis

Theoretical Analysis

The memoization algorithm has a best-case time complexity of $O(1)$ when $n = 1$, since the function immediately returns the result. Its worst-case time complexity is $O(n)$ because each subproblem is calculated only once and stored for future use. The space complexity is $O(n)$ due to the memoization table and recursion stack, which require additional memory as the input increases.

Experimental Analysis

For the input $n = 10$, the program produces a result of 3 minimum steps, following the sequence $10 \rightarrow 9 \rightarrow 3 \rightarrow 1$. The program uses memoization to store previously calculated results, preventing repeated computations and improving efficiency. Although larger inputs may require more processing time and memory, the stored results help maintain efficient performance.

Table 12-2. Memoization Implementation

Screenshot	ILO_B_Bottom-Up.cpp > ...
	1 #include <iostream>
	2 #include <vector>
	3 #include <algorithm>
	4
	5 using namespace std;
	6
	7 int getMinSteps(int n) {
	8 if (n <= 1) return 0; // Trivial case for 0 or 1
	9
	10 vector<int> dp(n + 1);
	11 dp[1] = 0; // Base case
	12
	13 for (int i = 2; i <= n; i++) {
	14 dp[i] = 1 + dp[i - 1];
	15 if (i % 2 == 0) dp[i] = min(dp[i], 1 + dp[i / 2]);
	16 if (i % 3 == 0) dp[i] = min(dp[i], 1 + dp[i / 3]);
	17 }
	18
	19 return dp[n];
	20 }
	21
	22 int main() {
	23 int n = 10;
	24 cout << "Minimum steps to reach " << n << " is: " << getMinSteps(n) << endl;
	25 return 0;
	26 }
	27

Output	Minimum steps to reach 10 is: 3
Analysis	**Theoretical Analysis** The bottom-up dynamic programming algorithm has a best-case time complexity of $O(1)$ when $n \leq 1$, since the function immediately returns the result. Its worst-case time complexity is $O(n)$ because it calculates the minimum steps for each number from 2 to n. Its space complexity is $O(n)$ because the DP array stores the results for each number. --- **Experimental Analysis** For the input $n = 10$, the program produces 3 minimum steps, following the sequence $10 \rightarrow 9 \rightarrow 3 \rightarrow 1$. The program uses a bottom-up approach, calculating and storing results from smaller numbers before solving larger ones. This avoids repeated recursive calls and improves efficiency.

Table 12-3. Bottom-Up Dynamic Programming Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Create C++ code to implement various algorithmic strategies & ILO C: Create C++ code to solve problems using learned algorithmic strategies

---

Pseudocode (+ Solving Problem by Hand) - *Santiago*

```
FUNCTION countPaths(int matrix[M][N], int row, int col, int totalCost, int targetCost)

    IF row >= M OR col >= N
        RETURN 0

    totalCost = totalCost + matrix[row][col]

    IF row == M - 1 AND col == N - 1
        IF totalCost == targetCost
            RETURN 1
        ELSE
            RETURN 0
        ENDIF
    ENDIF

    SET right = countPaths(matrix, row, col + 1, totalCost, targetCost)
    SET down = countPaths(matrix, row + 1, col, totalCost, targetCost)

    RETURN right + down

ENDFUNCTION

BEGIN
    DECLARE matrix[M][N]
    INPUT targetCost

    SET result = countPaths(matrix, 0, 0, 0, targetCost)

    DISPLAY result
END
```

Solving Problem by Hand - *Santiago*

GIVEN MATRIX:

4	7	1	6
6	7	3	9
3	8	1	2
7	1	7	3

PATH 1.

Moves Right, Then Down

4 → 6 → 3 → 8 → 1 → 7 → 3

4 + 6 + 3 + 8 + 1 + 7 + 3

Total Cost = 32

PATH 2:

Moves Down, Then Right

4 → 6 → 3 → 8 → 1 → 7 → 3

4 + 6 + 3 + 8 + 1 + 7 + 3

Total Cost: 32

PATH 3:

Moves Right and Down Alternately

4 → 7 → 7 → 8 → 1 → 2 → 3

4 + 7 + 7 + 8 + 1 + 2 + 3

Total Cost: 32

Working C++ Code - *Santiago*

```cpp
#include <iostream>

const int SIZE = 4;

int countPaths(int matrix[SIZE][SIZE], int row, int col,
               int totalCost, int targetCost) {
    // Check if the position is outside the matrix
    if (row >= SIZE || col >= SIZE) {
        return 0;
    }

    // Add the current cell's cost
    totalCost += matrix[row][col];

    // Check if the destination is reached
    if (row == SIZE - 1 && col == SIZE - 1) {
        if (totalCost == targetCost) {
            return 1;
        }
        return 0;
    }

    // Move right and move down
    int right = countPaths(matrix, row, col + 1,
                           totalCost, targetCost);

    int down = countPaths(matrix, row + 1, col,
                          totalCost, targetCost);

    // Return the total number of valid paths
    return right + down;
}
```

```cpp
int main() {
    int matrix[SIZE][SIZE] = {
        {4, 7, 1, 6},
        {6, 7, 3, 9},
        {3, 8, 1, 2},
        {7, 1, 7, 3}
    };

    int targetCost;

    std::cout << "Matrix:" << std::endl;

    for (int i = 0; i < SIZE; i++) {
        for (int j = 0; j < SIZE; j++) {
            std::cout << matrix[i][j] << "\t";
        }
        std::cout << std::endl;
    }

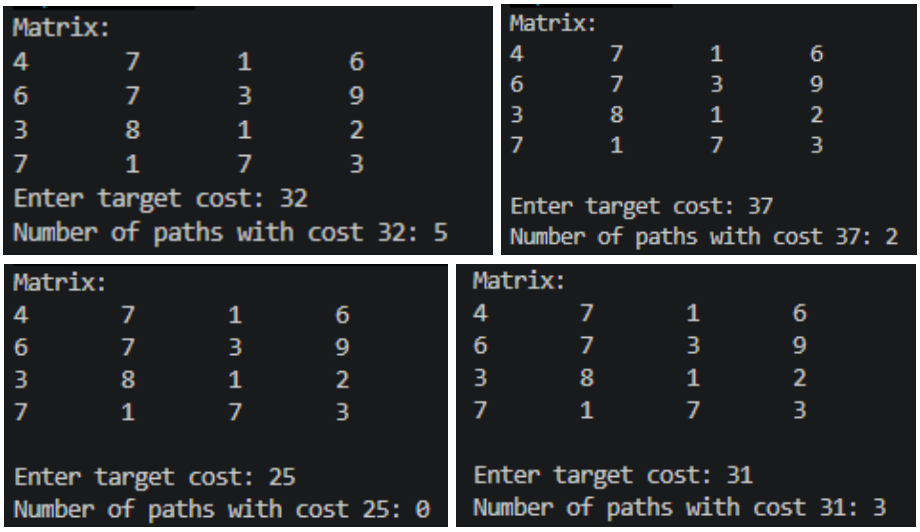
    std::cout << "\nEnter target cost: ";
    std::cin >> targetCost;

    int result = countPaths(matrix, 0, 0, 0, targetCost);

    std::cout << "Number of paths with cost "
              << targetCost << ": " << result << std::endl;

    return 0;
}
```

Output (Screenshots of demonstration) - *Santiago*



Theoretical Analysis - *Santiago*

The program uses recursion to explore all possible paths from the first cell to the destination cell. Since it can move only right or down, each recursive call explores two possible directions. In a general $M \times N$ matrix, the number of possible paths is $\binom{M+N-2}{M-1}$. Therefore, the recursive approach may take exponential time as the matrix grows. The recursion stack uses $O(M + N)$ space because each path has at most $M + N - 1$ visited cells.

Experimental Analysis - *Santiago*

When the program runs, it displays the 4 × 4 matrix and asks the user to enter the target cost. It then explores possible paths using recursive calls and checks whether the total cost of each completed path matches the target. The program displays the number of matching paths.

Overall Analysis - *Santiago*

The code works by counting the number of possible paths from the first cell to the last cell of a 4 × 4 matrix that have a total cost equal to the target cost entered by the user. First, the program displays the matrix and asks the user to input the target cost. It then calls the countPaths() function, which uses recursion to explore the possible paths by moving one cell to the right or one cell down. Every time the function visits a cell, its value is added to

the total cost. If the position goes outside the matrix, the function returns 0 because that path is invalid. When the function reaches the last cell, it checks whether the total cost matches the target cost. If they match, it returns 1; otherwise, it returns 0. The function continues exploring the remaining possible paths and adds their results to determine the total number of valid paths. Finally, the program displays the number of paths that match the target cost.

Conclusion / Analysis - *Santiago*

In this activity, we learned how different algorithmic strategies can be applied through actual coding and problem-solving. From our answers, we saw recursion in action when exploring all possible paths in the matrix, showing how problems can be broken down into smaller subproblems. Brute force was demonstrated in linear search, where every element is checked until the target is found. Backtracking appeared in DFS traversal, where we explored paths step by step and returned when no further moves were possible. Greedy strategy was shown in selection sort, which always picks the smallest element without looking ahead. Divide-and-conquer was applied in merge sort, where the list was split into smaller parts and then combined. Finally, dynamic programming was highlighted in the “minimum steps to one” problem, where memoization and bottom-up approaches avoided repeated work by storing results.

By connecting these strategies to our outputs, we understood not only their definitions from the discussion but also how they behave in practice. For example, the memoization and bottom-up DP codes both solved the minimum steps problem efficiently, while the matrix path-counting code showed how recursion can handle complex conditions.Overall, this activity taught us that the choice of strategy directly affects efficiency and clarity, and practicing them through programming implementations helped us see their strengths and limitations more concretely.

---

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: ***Grammarly***

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: ***Grammar correction***

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Grammarly was used to correct grammar corrections and correct vocabulary usage; it did not replace original thinking, analysis, authorship, or completion of the required work.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.